



Delicias Temuco Restaurante

Nicolás Cayumán, Gabriel Gutiérrez y Ailyn Melillán

Programación 2

Profesor: Guido Mellado

Ayudante: Joaquín Cantero

Ingeniería Civil en Informática

Universidad Católica de Temuco

21/11/2025

Respecto a la anterior evaluación, realizamos las siguientes mejoras:

- Migración total a ORM (SQLAlchemy).
- Implementación de CRUD modulares por entidad.
- Creación de DTOs para desacoplar UI y ORM.
- Nuevas pestañas en la interfaz (9 módulos nuevos).
- Uso de programación funcional (lambda, map, filter, reduce).
- Control real de stock mediante recetas (N:N).
- Implementación de PDF (carta y boleta).
- Gráficos estadísticos con Matplotlib.
- Carga masiva CSV y validaciones.

Diagrama de clases

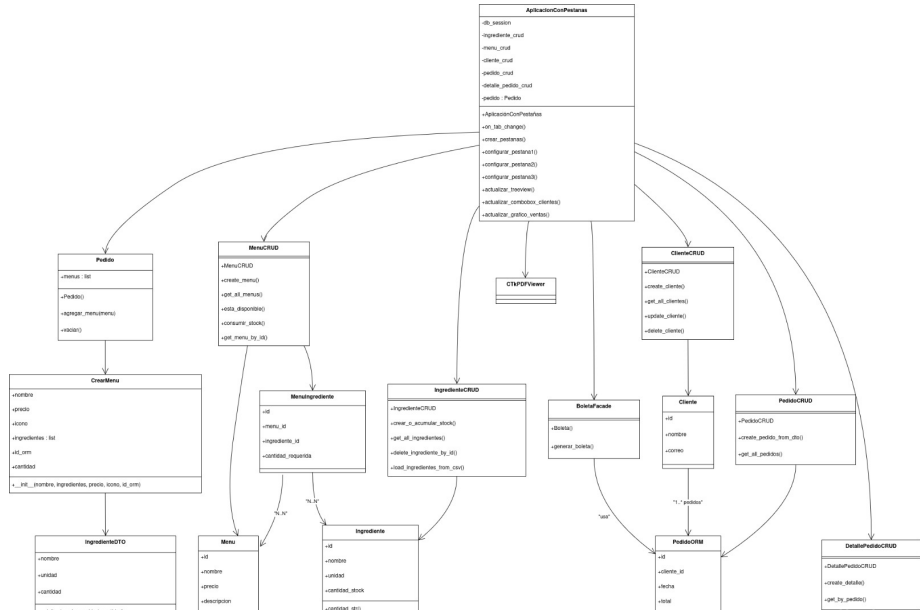
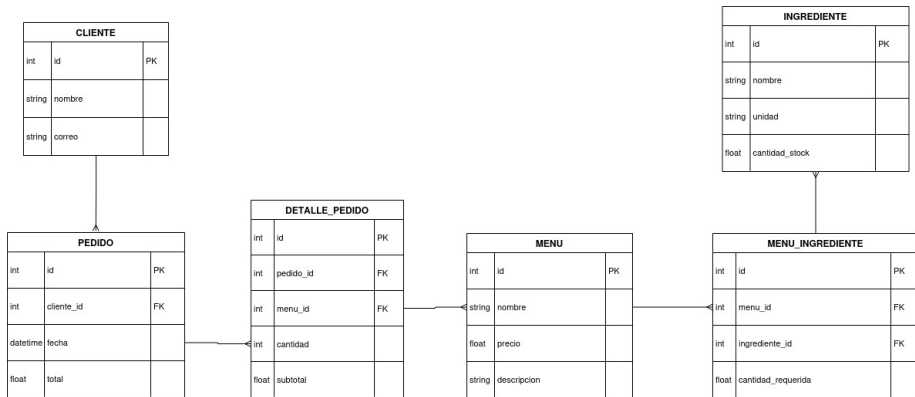


Diagrama MER



Solución propuesta

Se implementan cambios variados dentro de la aplicación en Python, que agrega más funciones para la gestión del restaurante:

- **Migración hacia ORM:** Se utilizan bases de datos para el manejo y registro de datos.
- **Gestión de clientes:** Se registran clientes para mayor control de ventas.
- **Generación automática de documentos:** boleta y carta en PDF con visualización integrada.
- **Gráficos** se implementa el registro de las compras por día semana mes y se generan graficos en relacion a esto para el control de ventas.
- **Edición de Menú:** Se permite desde la misma aplicación editar los menús del restaurante, permitiendo cambios de receta, precios, entre otros.

Flujo Principal del Sistema

El programa opera mediante una arquitectura basada en:

- **ORM (SQLAlchemy)** para la persistencia.
- **CRUDs modulares** para encapsular reglas de negocio.
- **DTOs** para aislar la interfaz del modelo.
- **CustomTkinter** para la interfaz multipestaña.

A continuación se detalla el flujo completo de ejecución.

1. Inicialización de la Aplicación

app.py es el punto de entrada del sistema:

- ❶ Se crean las tablas:
 - `create_db_and_tables()`
- ❷ Se genera una sesión de base de datos:
 - `self.db_session = SessionLocal()`
- ❸ Se inicializan los módulos CRUD:
 - `IngredienteCRUD`, `MenuCRUD`, `ClienteCRUD`, `PedidoCRUD`
- ❹ Se instancia el **Pedido (DTO)** que funcionará como carrito temporal.

2. Construcción de la Interfaz Gráfica (UI)

Al llamar a `crear_pestanas()` se generan las vistas:

- **Stock:** registro y actualización de ingredientes.
- **Carga CSV:** carga masiva mediante pandas.
- **Gestión de Menús:** creación, edición y eliminación.
- **Pedido:** selección de menús y validación de stock.
- **Boleta:** generación de PDF (Facade).
- **Carta:** PDF del catálogo de menús.
- **Clientes:** registro y edición.
- **Pedidos:** historial y control.
- **Gráficos:** estadísticas en tiempo real.

3. Construcción del Carrito y Lógica de Pedido

- ❶ La UI muestra cada menú como una **tarjeta** (DTO CrearMenu).
- ❷ El usuario agrega menús con un botón.
- ❸ Cada selección:
 - Valida stock disponible (`MenuCRUD.esta_disponible`).
 - Aumenta cantidad en el DTO.
 - Se recalcula el total con **reduce** + **lambda**.
- ❹ Se muestra una lista visual del pedido.

Este proceso puede realizarse varias veces antes de confirmar.

4. Confirmación del Pedido

Cuando el usuario confirma:

- ➊ Se crea un **PedidoORM** en BD.
- ➋ Por cada menú del carrito:
 - Se inserta un **DetallePedidoORM**.
 - Se descuenta el stock según receta (N:N).
- ➌ Se actualiza el historial de ventas.

BoletaFacade recibe el PedidoORM y genera un PDF con:

- Cliente
- Fecha
- Ítems
- Totales

5. Generación de PDF y Estadísticas

BoletaFacade.py:

- Construye una estructura de datos para la boleta.
- Renderiza el PDF.
- Lo muestra con CTkPDFViewer.

graficos.py:

- Extrae datos desde la BD con ORM.
- Genera gráficos de barras y torta en tiempo real.
- Integra estos gráficos dentro de la interfaz.

Uso del ORM (Ejemplos)

- Modelo de datos profesional.
- Relaciones entre entidades.
- Consultas mediante SQLAlchemy.
- Manejo de transacciones (commit/rollback).
- Uso de tabla intermedia (MenuIngrediente).

Incluido en EV3:

- `map` para poblar combobox.
- `filter` para filtrar menús disponibles.
- `reduce` para calcular total del pedido.
- `lambda` como funciones inline.

Ejemplos reales

map + lambda: etiquetas = list(map(lambda c: f"c.id - c.nombre",
clientes))

reduce: total = reduce(lambda t, m: t + m.precio*m.cantidad,
self.pedido.menus, 0)

Presentación de la interfaz visual

